

Continuous Control II: Max Entropy RL, SAC

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

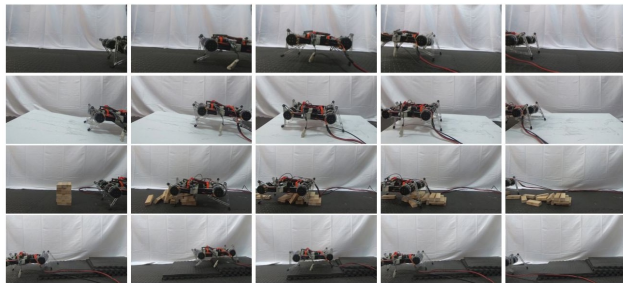
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 4, 2026

Continuous Control II

Max Entropy RL, SAC



Soft Actor-Critic — *Haarnoja, Zhou, Abbeel, Levine (2018)*

1. Recap
2. Learning Outcomes
3. Motivation
4. Background
5. Prior Work
6. Soft Actor-Critic
7. Experimental Results
8. Automatic Temperature Tuning
9. Summary
10. What's Next
11. References

Soft Actor-Critic: **Recap (DDPG & TD3)**

- ▶ DDPG [4] / TD3 [5] gave us **off-policy continuous control**: a replay buffer, target networks, and a deterministic actor $\mu_{\theta}(s)$

- ▶ DDPG [4] / TD3 [5] gave us **off-policy continuous control**: a replay buffer, target networks, and a deterministic actor $\mu_{\theta}(s)$
- ▶ But DDPG is **fragile**:
 - deterministic actor $\Rightarrow$ *no native exploration* — Gaussian noise has to be bolted on by hand, with its own scale and schedule to tune
 - brittle to hyperparameters and random seeds
 - mode collapse on multimodal Q -landscapes — commits to one of several equally good actions

- ▶ **Today — Soft Actor-Critic (SAC)** [1]: fix all three with a *stochastic* actor and an *entropy bonus* in the objective

$$J(\pi) = \sum_t \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

- ▶ Entropy drives *directed* exploration — no hand-tuned noise schedule; robust to seeds; captures multimodal optima
- ▶ **Everything from DDPG/TD3 carries over:** replay buffer, target networks, and TD3's clipped double-Q critic
- ▶ We close with **automatic temperature tuning** (SAC v2) — removing the last brittle hyperparameter

Scope note: “Continuous Control II” is the *max-entropy* branch. PPO already handles continuous actions on-policy; DDPG/TD3 are the off-policy *deterministic* branch; SAC is the off-policy *stochastic*, *entropy-regularised* branch.

Symbol	Meaning	Where you saw it
θ	actor parameters	REINFORCE, A2C/PPO, DDPG
ϕ	critic parameters	A2C/PPO, DDPG
$\pi_{\theta}(a s)$	<i>stochastic</i> actor (back today)	REINFORCE, A2C/PPO
$\mu_{\theta}(s), \sigma_{\theta}(s)$	mean and spread of that actor	REINFORCE (DDPG kept only μ_{θ})
$Q_{\phi}(s, a)$	<i>soft</i> action-value critic	DDPG (now with an entropy term)
ϕ^{-}	target critic parameters	DQN, DDPG
$\mathcal{D}$	replay buffer	DQN, DDPG
α	entropy <i>temperature</i> (new)	—
$\mathcal{H}(\pi(\cdot s))$	entropy of the actor at state s (new)	—
$\bar{\mathcal{H}}$	target entropy (new, SAC v2)	—

We said the noise would come back — here it is. For DDPG we wrote the actor as a Gaussian $\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}(s)^2)$, then *deleted* σ_{θ} to get the deterministic μ_{θ} . SAC puts σ_{θ} back and *learns* it. **Heads-up:** the SAC paper swaps the letters (ϕ actor, θ critic); we keep the course convention — only the letters differ, every equation is identical.

- ▶ Define entropy $\mathcal{H}$ and explain why the per-action bonus is $-\alpha \log \pi$ (exploration, robustness, multimodality)
- ▶ Say what “**soft**” means in one sentence — $\max$ replaced by $\alpha \log \sum \exp(\cdot/\alpha)$ — and apply that substitution to a value, a policy, and a Bellman backup
- ▶ Walk through soft policy iteration (evaluate $\leftrightarrow$ improve) and say why the standard policy-iteration guarantees still hold
- ▶ Derive the SAC critic and (reparameterised) actor objectives, including the squashed-Gaussian Jacobian correction
- ▶ Explain how $\text{SAC v2} = \text{SAC v1} + \text{automatic temperature tuning}$ (the clipped double-Q was already in v1), and what reward-scale sensitivity has to do with it
- ▶ Identify SAC’s remaining limitations and place it in the algorithm family (REDQ / DroQ / TQC)

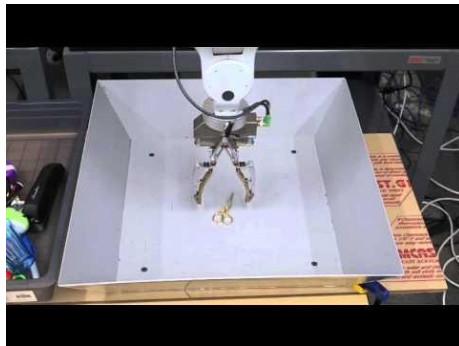
- ▶ Number of times the agent must interact with the environment in order to learn a task
- ▶ Good sample complexity is the first prerequisite for successful skill acquisition.
- ▶ Learning skills in the real world can take a substantial amount of time
 - can get damaged through trial and error

- ▶ “Learning Hand-Eye Coordination for Robotic Grasping with Deep Learning and Large-Scale Data Collection”, Levine et al., 2016 [8]
 - up to 14 robot arms learning to grasp in parallel
 - objects started being picked up at around 20,000 grasps



<https://spectrum.ieee.org/automaton/robotics/artificial-intelligence/google-large-scale-robotic-grasping-project>

Observing the behavior of the robot after over 800,000 grasp attempts, which is equivalent to about 3000 robot-hours of practice, we can see the beginnings of intelligent reactive behaviors.



https://www.youtube.com/watch?v=cXaic_k80uM

- **Off-policy is the route to sample efficiency.** On-policy methods (PPO, A2C) discard data after each update; off-policy ones reuse it via a replay buffer $\mathcal{D}$ — SAC, like DDPG, is built on this.

Value-based (DQN):

$$\text{Q-Learning: } Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

$$\text{DQN [6]: } \mathcal{L}(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(r + \gamma \max_{a'} Q_{\phi^-}(s', a') - Q_{\phi}(s, a))^2 \right]$$

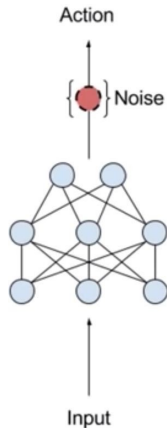
- Replay buffer + target network ϕ^- stabilise off-policy TD — but $\max_{a'}$ doesn't scale to continuous actions

Actor-critic (A2C/PPO, DDPG):

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a | s) \hat{A}(s, a) \right] \quad \text{DDPG: } \nabla_{\theta} Q_{\phi}(s, \mu_{\theta}(s))$$

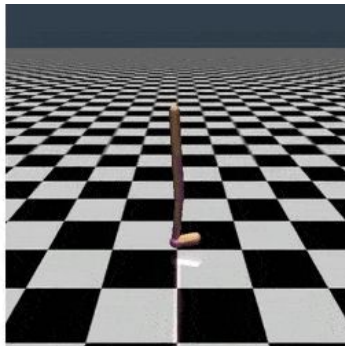
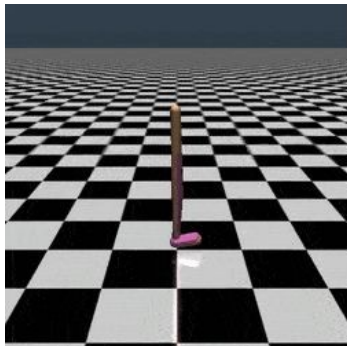
- DDPG/TD3 [4][5]: deterministic actor μ_{θ} — replay + targets reused, but exploration is just injected noise

- ▶ DDPG = DQN + DPG (Lillicrap et al., 2015)
 - off-policy actor-critic method that learns a deterministic policy in continuous domain
 - exploration noise added to the deterministic policy when select action
 - difficult to stabilize and brittle to hyperparameters (Duan et al., 2016, Henderson et al., 2017)
 - unscalable to complex tasks with high dimensions (Gu et al., 2017)



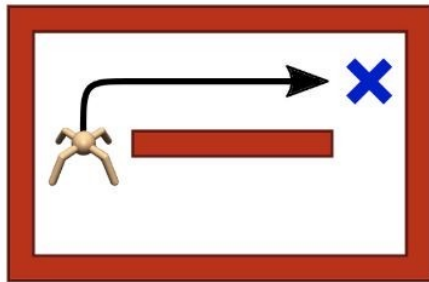
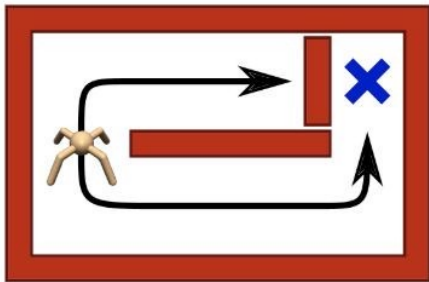
<https://www.youtube.com/watch?v=zR11FLZ-09M>

- ▶ Training is sensitive to randomness in the environment, initialization of the policy and the algorithm implementation



<https://gym.openai.com/envs/Walker2d-v2/>

- ▶ Knowing only one way to act makes agents vulnerable to environmental changes that are common in the real-world



<https://bair.berkeley.edu/blog/2017/10/06/soft-q-learning/>

Conventional RL Objective — Expected Reward

$$\sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t)]$$

Maximum Entropy RL Objective — Expected Reward + Entropy of the Policy

$$\sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))]$$

- ▶ In words: “collect reward, **and** stay as undecided as you can afford to”
 - ▶ α (the **temperature**) sets the exchange rate: how much reward is one unit of undecidedness worth?
- ⇒ Next two slides: what *is* $\mathcal{H}$, and why does adding it make everything “soft”?

What Is Entropy? (and why the minus sign)

Step 1 — how surprising is a single outcome? Define the *surprise* of seeing x as $-\log P(x)$:

- ▶ $P(x) = 1$ (certain) $\Rightarrow -\log 1 = 0$: no surprise at all
- ▶ $P(x)$ tiny (rare) $\Rightarrow -\log P(x)$ large: very surprising
- ▶ **Why a log?** For independent events $P(x, y) = P(x)P(y)$, and $-\log P(x, y) = -\log P(x) - \log P(y)$. Surprises should *add up*, and log is what turns multiplying into adding
- ▶ **Why the minus?** Probabilities are at most 1, so $\log P(x)$ is negative. The minus just makes surprise a positive number

Step 2 — entropy is the average surprise:

$$\mathcal{H}(P) = \mathbb{E}_{x \sim P} \left[\underbrace{-\log P(x)}_{\text{surprise of } x} \right]$$

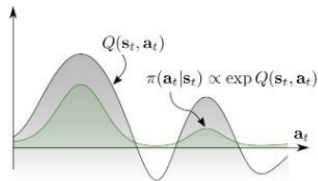
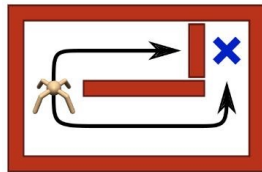
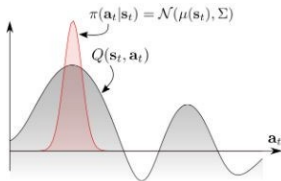
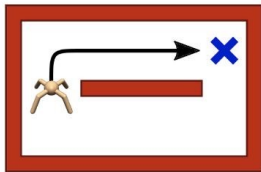
Distribution	Entropy	Reading
always the same outcome	0	fully predictable
fair coin	$\log 2 \approx 0.69$	one bit of doubt
uniform over n outcomes	$\log n$ (the largest possible)	maximally undecided
Gaussian $\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{2} \log(2\pi e \sigma^2)$	grows with the spread σ

- ▶ $\mathcal{H}$ is an *average* over actions — but the agent only ever has the *one* action it sampled. Just unfold the definition:

$$\alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t)) = \mathbb{E}_{\mathbf{a}_t \sim \pi} [-\alpha \log \pi(\mathbf{a}_t | \mathbf{s}_t)]$$

- ▶ So per sampled action the bonus is $-\alpha \log \pi(\mathbf{a}_t | \mathbf{s}_t)$: **the agent is paid extra for taking an action it thought was unlikely.** *This one term is the only thing SAC adds* — it appears in every equation for the rest of the lecture
- ▶ Why this fixes DDPG's three problems:
 - **Exploration.** Staying spread out is now *paid for by the objective*, and how much to spread is learned per state — not a noise schedule you hand-tune
 - **Multimodality.** Two equally good actions? Collapsing onto one costs entropy, so the actor keeps both
 - **Robustness.** More than one way to act means an alternative survives when the environment shifts
- ▶ **Sanity check:** set $\alpha \rightarrow 0$ and the bonus vanishes — what is left is exactly the ordinary RL objective from RL Foundations. MaxEnt RL is a *strict generalisation*

- MaxEnt RL agent can capture different modes of optimality to improve robustness against environmental changes



<https://bair.berkeley.edu/blog/2017/10/06/soft-q-learning/>

So What Does “Soft” Actually Mean?

- ▶ $\max_a Q(s, a)$ reports the top score and *throws away everything else*
- ▶ **Soft max** = replace it with a smooth version. We *define*:

$$\text{soft max}_a Q(s, a) = \alpha \log \sum_a \exp\left(\frac{1}{\alpha} Q(s, a)\right)$$

Reminder: $\exp(x)$ is just another way of writing e^x .

- ▶ **Near-ties get a bonus** — having a good backup plan is worth something
- ▶ **A clear winner gets essentially nothing:** it reduces to the plain max
- ▶ **It is smooth and differentiable**, unlike max, which jumps the instant the ranking flips
- ▶ **The temperature α is the dial:** $\alpha \rightarrow 0$ recovers the hard max exactly (the largest term swamps the sum); large α washes the differences out and just averages

For continuous actions the sum becomes $\int_{\mathcal{A}} \cdots d\mathbf{a}$ — nothing else changes.

Two actions, temperature $\alpha = 1$, so soft max = $\log(e^{Q_1} + e^{Q_2})$:

Q values	hard max	soft max	the gap
10, 0	10	10.00005	0.00005
10, 8	10	10.13	0.13
10, 9.5	10	10.47	0.47
10, 10	10	10.69	0.69

- **Yes, the soft max is bigger than the max — always.** You are adding a positive quantity to the winner's score, and the closer the runner-up, the bigger it gets. **That gap is the entropy bonus**
- Bottom row: the gap is $\log 2 = 0.69$, exactly the entropy of a fair coin — the payment for genuinely having two options. A runner-up 10 points behind buys you nothing
- **The dial, concretely.** On that perfect tie the soft max is exactly $\max + \alpha \log 2$:
 - $\alpha = 0 \Rightarrow 10.00$ (the hard max) $\alpha = 1 \Rightarrow 10.69$ $\alpha = 2 \Rightarrow 11.39$

- The hard max asks *which single action is best?* The soft max asks a different question: *which distribution over actions is best, if I am paid α per unit of entropy?*

$$\underbrace{\alpha \log \sum_a \exp\left(\frac{1}{\alpha} Q(s, a)\right)}_{\text{soft max}} = \max_{\pi} \left\{ \underbrace{\mathbb{E}_{a \sim \pi} [Q(s, a)]}_{\text{expected value}} + \underbrace{\alpha \mathcal{H}(\pi(\cdot | s))}_{\text{entropy bonus}} \right\} \quad \text{achieved by} \quad \pi(a | s) \propto \exp\left(\frac{1}{\alpha} Q(s, a)\right)$$

- **You have met this distribution before.** “ α ” just means *proportional to* — written out in full, $\pi(a | s) = \frac{\exp(Q(s, a)/\alpha)}{\sum_{a'} \exp(Q(s, a')/\alpha)}$, which is the **softmax** from classification with Q -values as the scores and α as the temperature. The bottom line is the normaliser that makes it sum to 1; the log-sum-exp is that same normaliser read as a value
- $\alpha \rightarrow 0$: softmax $\rightarrow$ one-hot $\Rightarrow$ the greedy deterministic policy of Q-learning and DQN
 - α large: softmax $\rightarrow$ uniform $\Rightarrow$ random flailing
- $\Rightarrow$ **One idea wearing three hats:** the soft max (a *value*), the softmax policy (a *distribution*) and “reward + $\alpha \mathcal{H}$ ” (an *objective*) are the same statement

	Hard (so far)	Soft (today)
value	$V(s) = \max_a Q(s, a)$	$V_{\text{soft}}(s) = \alpha \log \sum_a \exp(\frac{1}{\alpha} Q(s, a))$
policy	greedy: $\pi(s) = \arg \max_a Q(s, a)$	softmax: $\pi(a s) \propto \exp(\frac{1}{\alpha} Q(s, a))$
TD target	$Q \leftarrow r + \gamma \max_{a'} Q(s', a')$	$Q \leftarrow r + \gamma \text{soft max}_{a'} Q(s', a')$
objective	$\sum_t \mathbb{E}[r_t]$	$\sum_t \mathbb{E}[r_t + \alpha \mathcal{H}(\pi(\cdot \mathbf{s}_t))]$

- ▶ Every row is the *same single substitution*. Nothing else built so far changes: replay buffer, target networks, TD updates and clipped double-Q all carry over untouched
- ▶ Set $\alpha \rightarrow 0$ and every right-hand entry collapses back to the left — “soft” costs you nothing, it only adds a dial

The TD-target row is the one that changes your code. Every value method since DQN has bootstrapped with $\max_{a'} Q(s', a')$ inside the TD target. **In MaxEnt RL that max is replaced by the soft max — nothing else about TD learning changes.** And once you have an actor to sample from, the soft max is computed as $\mathbb{E}_{a' \sim \pi}[Q(s', a') - \alpha \log \pi(a' | s')]$, so no exponential ever appears in the code.

- **Soft Q-Learning** (Haarnoja et al., 2017) [7] — Q-learning with the dictionary applied
 - off-policy, under the MaxEnt RL objective
 - learns Q_{soft} directly; no separate actor network
 - acting is no longer $\arg \max$ — you *sample* from the softmax policy $\pi \propto \exp(\frac{1}{\alpha} Q_{\text{soft}})$
 - **the catch**: with continuous actions, sampling from that distribution is itself hard — no closed form, and no actor network to ask. This is exactly the gap SAC closes

Standard Q-learning, for contrast — note the hard max:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_l r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)$$

Soft version — the only change is $\max \rightarrow \text{soft max}$:

$$Q_{\text{soft}}(\mathbf{s}_t, \mathbf{a}_t) \leftarrow r_t + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p_{\mathbf{s}}} [V_{\text{soft}}(\mathbf{s}_{t+1})] \quad V_{\text{soft}}(\mathbf{s}_t) \leftarrow \alpha \log \int_{\mathcal{A}} \exp(\frac{1}{\alpha} Q_{\text{soft}}(\mathbf{s}_t, \mathbf{a}')) d\mathbf{a}'$$

This is the “value” row of the dictionary substituted into the Q-learning update, nothing more. The $\int$ instead of $\sum$ is only because the actions are continuous. (*The learning rate α_l here is unrelated to the temperature α — clashing but standard notation.*)

- ▶ One of the most efficient model-free algorithms
 - SOTA off-policy
 - well suited for real world robotics learning
- ▶ Can learn stochastic policy on continuous action domain
- ▶ Robust to noise
- ▶ **Ingredients:**
 - Actor-critic architecture with separate policy and value function networks
 - Off-policy formulation to reuse previously collected data for efficiency
 - Entropy-constrained objective to encourage stability and exploration

- ▶ Classic **policy iteration**: alternate two steps until nothing moves
 1. **evaluation**: given π , work out its Q
 2. **improvement**: given Q , make π greedier
 - ▶ **Soft policy iteration is the same loop with the dictionary applied** — “greedier” becomes “closer to the softmax of Q ”, and the values carry the entropy bonus
 - ▶ The next two slides do one step each. The only question worth asking is: *does swapping max for soft max break the guarantees of ordinary policy iteration?*
- ⇒ **It does not.** And once we know that, we can go build the deep version (SAC) without worrying

- **Goal:** given a fixed π , compute its value *under the MaxEnt objective*. Same Bellman backup as usual, with the entropy bonus folded into the value:

$$\mathcal{T}^\pi Q(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V(\mathbf{s}_{t+1})]$$

$$V(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi} [Q(\mathbf{s}_t, \mathbf{a}_t) - \alpha \log \pi(\mathbf{a}_t | \mathbf{s}_t)] = \mathbb{E}_\pi [Q] + \alpha \mathcal{H}(\pi(\cdot | \mathbf{s}_t))$$

- **Does repeating this converge?** Yes — and for a boring reason. Compare with the standard case:
 - The standard proof only needed the backup to *shrink differences* between value estimates by a factor γ each sweep. That is what makes repeated application settle on one answer
 - The single new term $-\alpha \log \pi$ does not depend on Q at all. It is just **an extra piece of reward**, and that proof never cared what the reward was
- ⇒ **Nothing new to prove.** Soft policy evaluation is ordinary policy evaluation on the reward $r + \alpha \mathcal{H}$

- ▶ **Standard RL:** improve π by making it greedy, $\pi \leftarrow \arg \max_a Q(s, a)$
- ▶ **Today:** improve π by moving it toward the *softmax* of Q , $\pi \propto \exp(\frac{1}{\alpha} Q)$ — the distribution we showed is optimal for “value + α entropy”
- ▶ **One practical snag:** $\exp(\frac{1}{\alpha} Q)$ can come out *any shape at all*, and no network can output an arbitrary distribution. Whatever actor we build will only produce distributions from some limited family — call it Π
- ▶ So we settle for *the closest member of Π* , and “closest” between two distributions means the KL divergence you met in TRPO:

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}}\left(\pi'(\cdot | \mathbf{s}_t) \parallel \underbrace{\exp(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(\mathbf{s}_t, \cdot)) / Z(\mathbf{s}_t)}_{\text{the softmax policy we are aiming at}}\right)$$

Π stays abstract for now — the theory works for *any* choice; we pick a concrete one when we build SAC. $Z(\mathbf{s}_t)$ just makes the target sum to one, and drops out of the arg min because it does not depend on π' .

- ▶ **Does this actually improve anything?** Yes, and again for a simple reason: π_{old} is itself one of the candidates in the arg min. So π_{new} is by construction no worse — the soft value can only go up

- ▶ **Soft policy iteration** = alternate the two steps: evaluate $\longleftrightarrow$ improve
 - ▶ **Why it lands somewhere:** improvement means the values only ever go *up*; rewards and entropy are bounded so they cannot go up forever. Something that only climbs and cannot climb past a ceiling has to settle — and where it settles is the best MaxEnt policy the network family can express
 - ▶ **But we cannot run this loop as written.** It assumes we can sweep over *every* state and action — fine for a small gridworld, hopeless for a robot with continuous joints
- ⇒ Replace the two exact steps with two **neural networks trained by gradient descent**:
- evaluation $\rightarrow$ a critic Q_ϕ fitting the soft Bellman target
 - improvement $\rightarrow$ an actor π_θ taking gradient steps on that KL

That is Soft Actor-Critic.

The two “does it work” arguments above are Lemmas 1–2 in the paper [1], and the last one is its Theorem 1 — look there if you want them stated formally.

Symbol	Meaning	Example
$Q_\phi(\mathbf{s}_t, \mathbf{a}_t)$	parameterized soft Q -function (the critic)	neural network
$\pi_\theta(\mathbf{a}_t \mathbf{s}_t)$	parameterized tractable policy (the actor)	Gaussian, μ_θ and σ_θ from a network
$J_Q(\phi), \hat{\nabla}_\phi J_Q(\phi)$	critic objective & its stochastic gradient	minimize soft Bellman residual
$J_\pi(\theta), \hat{\nabla}_\theta J_\pi(\theta)$	actor objective & its stochastic gradient	KL to $\exp(Q/\alpha)$, reparameterized

This is where we finally pick Π : the actor's family is the **Gaussians** — one mean and one spread per action dimension. That is the concrete choice policy improvement was left waiting on. Otherwise these are the same two networks as DDPG: the critic now carries the entropy term, and the actor outputs a *distribution* $(\mu_\theta, \sigma_\theta)$ instead of a single action μ_θ .

- **Critic — soft Q-function:** minimise the squared soft Bellman error, exactly as in DQN/DDPG. ϕ^- is the slow-moving target copy of the critic weights (as in DQN)

$$J_Q(\phi) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \mathcal{D}} \left[\frac{1}{2} \left(Q_\phi(\mathbf{s}_t, \mathbf{a}_t) - \left(r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V_{\phi^-}(\mathbf{s}_{t+1})] \right) \right)^2 \right]$$

$$V_{\phi^-}(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi_\theta} [Q_{\phi^-}(\mathbf{s}_t, \mathbf{a}_t) - \alpha \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)]$$

$$\hat{\nabla}_\phi J_Q(\phi) = \nabla_\phi Q_\phi(\mathbf{s}_t, \mathbf{a}_t) \left(Q_\phi(\mathbf{s}_t, \mathbf{a}_t) - \left(r(\mathbf{s}_t, \mathbf{a}_t) + \gamma [Q_{\phi^-}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \alpha \log \pi_\theta(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})] \right) \right)$$

- ⇒ **This is the soft max sitting in the TD target.** DQN bootstrapped with $\max_{a'} Q(s', a')$; here the same slot holds $\mathbb{E}_{a' \sim \pi_\theta} [Q_{\phi^-} - \alpha \log \pi_\theta]$, which is the soft max evaluated with the actor. Cover the red and this is the DDPG critic update, unchanged.

- **Actor:** the KL from the previous slide, $\arg \min_{\pi' \in \Pi} D_{\text{KL}}(\pi' \parallel \exp(\frac{1}{\alpha} Q_\phi)/Z)$. Multiply through by α and drop Z (constant in π') and it becomes something you can just call `.backward()` on:

$$J_\pi(\theta) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}, \mathbf{a}_t \sim \pi_\theta} [\alpha \log \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t) - Q_\phi(\mathbf{s}_t, \mathbf{a}_t)]$$

- **In words:** *climb the critic* (the $-Q_\phi$ term) *but stay spread out* (the $+\alpha \log \pi_\theta$ term, which is $-\alpha \mathcal{H}$). DDPG's actor had only the first of those
- **Where the pieces came from:**
- the arg min over Π is soft policy improvement, now done by *gradient descent* instead of exactly
 - multiplying by α is cosmetic — it does not move the arg min
 - $Z(\mathbf{s}_t)$ is the intractable normaliser, and this is precisely where it disappears: it does not depend on π' , so its gradient is zero. **We never have to compute it**

⇒ One snag remains before we can actually differentiate this — next slide

- ▶ **The actor is one network, and it does not output an action.** Feed it a state and it returns **two vectors**, each with one number per action dimension:

- $\mu_\theta(\mathbf{s})$ — the **mean**: where to aim
- $\sigma_\theta(\mathbf{s})$ — the **spread**: how much to vary around that aim

That is the entire output. For a 6-joint arm, the actor emits 6 means and 6 spreads.

- ▶ **An action is then built from them in three steps:**

$$\underbrace{\epsilon \sim \mathcal{N}(0, I)}_{1. \text{ draw noise}}$$

$$\underbrace{\mathbf{u} = \mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s}) \odot \epsilon}_{2. \text{ scale and shift it}}$$

$$\underbrace{\mathbf{a} = \tanh(\mathbf{u})}_{3. \text{ squash into range}}$$

- ▶ **Why the tanh?** Step 2 can return *any* real number — +500 is a perfectly legal Gaussian sample. A robot joint cannot accept a torque of 500. tanh folds the whole real line into $(-1, 1)$, so every action is in the actuator's range *by construction*, and unlike clipping it stays differentiable everywhere

- ▶ **At evaluation time** drop step 1 and act with $\mathbf{a} = \tanh(\mu_\theta(\mathbf{s}))$ — the mean, no noise

⇒ $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ is the *distribution* these three steps induce. μ_θ and σ_θ are the only things the network learns

- ▶ **The snag:** the actor objective is an average over actions *sampled from the very network we are differentiating*. You cannot backpropagate through a random draw — “sample from π_θ ” has no derivative with respect to θ
- ▶ **The fix is already in the three steps.** Look at where the randomness entered: step 1, as $\epsilon \sim \mathcal{N}(0, I)$ — a *fixed* noise source that does not depend on θ at all. Steps 2 and 3 are just multiply, add and tanh: an ordinary differentiable function
 - so the randomness is an **input**, not an operation we have to differentiate through
 - gradients flow from the action back into μ_θ and σ_θ as usual
- ▶ This is the **reparameterization trick**. We give the three steps a single name:

$$\mathbf{a}_t = f_\theta(\epsilon_t; \mathbf{s}_t) = \tanh(\mu_\theta(\mathbf{s}_t) + \sigma_\theta(\mathbf{s}_t) \odot \epsilon_t), \quad \epsilon_t \sim \mathcal{N}(0, I)$$

f_θ is not a new network — it is just shorthand for “run the actor, then steps 2 and 3”. Wherever you see $f_\theta(\epsilon_t; \mathbf{s}_t)$ from here on, read it as “the action”.

- ▶ Every SAC equation needs one number: $\log \pi_{\theta}(\mathbf{a} \mid \mathbf{s})$, *how likely was the action we just played*. The Gaussian gives us the likelihood of $\mathbf{u}$ — but we did not play $\mathbf{u}$, we played $\mathbf{a} = \tanh(\mathbf{u})$. **Those are two different numbers.**
- ▶ **Why they differ.** A density is *probability per unit width*. $\tanh$ crushes wide stretches of $\mathbf{u}$ into narrow slivers of $\mathbf{a}$: the probability inside a stretch is unchanged, but the width is not — so the density goes up.

this range of $\mathbf{u}$	becomes this range of $\mathbf{a}$	density multiplied by
[0, 1]	[0.00, 0.76]	$\times 1.3$
[1, 2]	[0.76, 0.96]	$\times 4.9$
[3, 4]	[0.995, 0.999]	$\times 234$

⇒ Whatever chance $\mathbf{u}$ had of landing in [3, 4], $\mathbf{a}$ has *exactly* that chance of landing in a sliver $234\times$ narrower. Same probability, $234\times$ less room. **Near the edge of the action range the true density is hundreds of times the Gaussian's.**

- **The fix.** Divide by how much the width shrank. That factor is just the slope of $\tanh$, namely $1 - \tanh^2(u)$ — and in logs, dividing becomes subtracting:

$$\log \pi_{\theta}(\mathbf{a} \mid \mathbf{s}) = \underbrace{\log \mathcal{N}(\mathbf{u}; \mu_{\theta}, \sigma_{\theta})}_{\text{plain Gaussian — easy}} - \underbrace{\sum_i \log(1 - \tanh^2(u_i))}_{\text{the correction}}$$

Check it against the table: at $u = 3.4$ the slope is 0.0044, so the correction is $-\log(0.0044) = 5.4$ — a factor of $e^{5.4} \approx 225$.

- **Why you cannot skip it.** The correction is **not a constant**: near 0 in the middle of the range, around 5.4 at the edges. Leave it out and two things break at once:
- saturated actions look *less* likely than they really are, so the agent collects an inflated bonus $-\alpha \log \pi_{\theta}$ simply for slamming the actuator to its limit
 - the reported entropy comes out too high, so SAC v2's tuner believes its target is already met and drives α down — quietly killing the exploration the whole method exists for

⇒ *One line of code, and a classic silent bug if you forget it.*

- ▶ Substituting $\mathbf{a}_t = f_\theta(\epsilon_t; \mathbf{s}_t)$ into the actor objective, the expectation is now over *noise* — which does not depend on θ — so we can differentiate straight through it:

$$J_\pi(\theta) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}, \epsilon_t \sim \mathcal{N}}[\alpha \log \pi_\theta(f_\theta(\epsilon_t; \mathbf{s}_t) | \mathbf{s}_t) - Q_\phi(\mathbf{s}_t, f_\theta(\epsilon_t; \mathbf{s}_t))]$$

$$\hat{\nabla}_\theta J_\pi(\theta) = \nabla_\theta \alpha \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) + \left(\nabla_{\mathbf{a}_t} \alpha \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) - \nabla_{\mathbf{a}_t} Q_\phi(\mathbf{s}_t, \mathbf{a}_t) \right) \nabla_\theta f_\theta(\epsilon_t; \mathbf{s}_t)$$

- ⇒ The gradient flows *through the critic into the actor*, exactly like DDPG's $\nabla_\theta Q_\phi(s, \mu_\theta(s))$ — now for a stochastic actor instead of a deterministic one

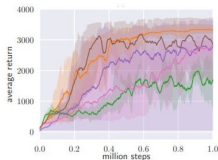
In code this is one line: sample ϵ , build the action, evaluate $\alpha \log \pi_\theta - Q_\phi$, call `.backward()`.

Red = the **two** things that differ from TD3: the actor is *stochastic*, so actions are **sampled** from it, and the TD target carries the *entropy* term.

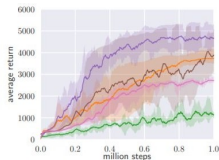
Shorthand: $Q_{\phi^-}^{\min}$ means $\min_{i=1,2} Q_{\phi_i^-}$. α is a **fixed hyperparameter** here. λ_Q, λ_π are **learning rates** (Adam step sizes) for critic and actor; τ is the target-network mixing rate from DDPG.

```
Initialize actor  $\theta$ , critics  $\phi_1, \phi_2$ ; targets  $\phi_i^- \leftarrow \phi_i$ ; replay  $\mathcal{D} \leftarrow \emptyset$ ; choose  $\alpha$ 
for each iteration do
  for each environment step do
    |  $\mathbf{a}_t \sim \pi_\theta(\cdot | \mathbf{s}_t)$  (sampled — no injected noise);  $\mathbf{s}_{t+1} \sim p$ ;  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})\}$ 
  end
  for each gradient step do
    | sample minibatch from  $\mathcal{D}$ ;  $\tilde{\mathbf{a}}_{t+1} \sim \pi_\theta(\cdot | \mathbf{s}_{t+1})$  (sampled from the current actor)
    |  $y = r_t + \gamma(Q_{\phi^-}^{\min}(\mathbf{s}_{t+1}, \tilde{\mathbf{a}}_{t+1}) - \alpha \log \pi_\theta(\tilde{\mathbf{a}}_{t+1} | \mathbf{s}_{t+1}))$ 
    |  $\phi_i \leftarrow \phi_i - \lambda_Q \hat{\nabla}_{\phi_i} [\frac{1}{2}(Q_{\phi_i}(\mathbf{s}_t, \mathbf{a}_t) - y)^2]$ ,  $i \in \{1, 2\}$ 
    |  $\theta \leftarrow \theta - \lambda_\pi \hat{\nabla}_\theta J_\pi(\theta)$  ( $J_\pi$  critic term uses  $Q_{\phi}^{\min}$ )
    |  $\phi_i^- \leftarrow \tau \phi_i + (1 - \tau) \phi_i^-$ ,  $i \in \{1, 2\}$ 
  end
end
```

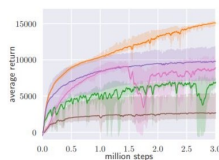
Black = unchanged from TD3: replay buffer, twin critics, clipped double-Q target, Polyak target critics. **But the actor is not TD3's** — it outputs a distribution $(\mu_\theta, \sigma_\theta)$, never a bare action, which is why SAC needs *no target actor* and *no smoothing noise*: TD3 needed both only because its actor was deterministic.



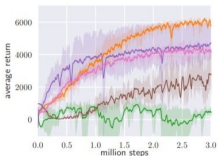
(a) Hopper-v1



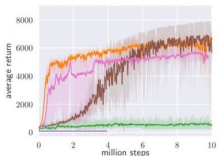
(b) Walker2d-v1



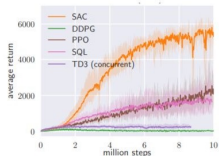
(c) HalfCheetah-v1



(d) Ant-v1



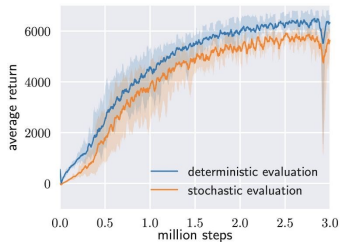
(e) Humanoid-v1



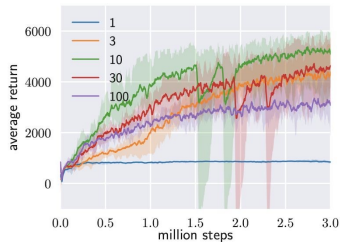
(f) Humanoid (rllab)

Tasks: OpenAI gym continuous-control suite + the 21-dim rllab Humanoid (exceptionally hard for off-policy methods). **Baselines:** DDPG [4], SQL [7], PPO, TD3 (concurrent) [5].

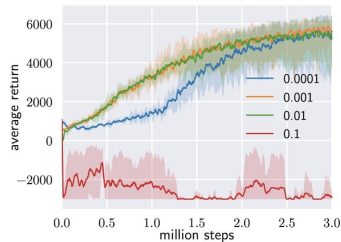
⇒ SAC matches or beats DDPG/PPO/TD3/SQL on every benchmark, and is the *only* off-policy method to make real progress on Humanoid (rllab) — the headline win.



(a) Evaluation



(b) Reward Scale



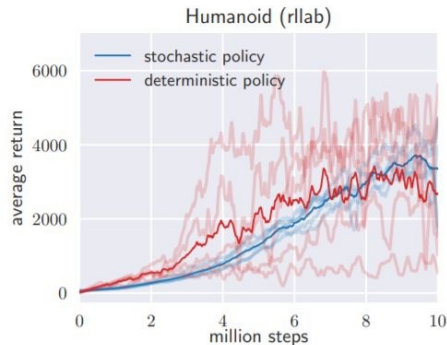
(c) Target Smoothing Coefficient (τ)

Reward scale (left): performance is *dramatically* sensitive — too small $\rightarrow$ no exploration signal, too large $\rightarrow$ near-deterministic; the right scale is task-specific. **Target smoothing τ** (right): mild sensitivity, the usual stability/speed trade-off.

$\Rightarrow$ Reward scale is just a hand-tuned proxy for the entropy temperature α — *this* is the problem automatic temperature tuning solves.

<https://arxiv.org/abs/1801.01290>

- ▶ How does policy stochasticity and entropy maximization affect performance?
 - ▶ Compared with a deterministic SAC variant (no entropy) that closely resembles DDPG
- ⇒ Stochastic+entropy is far more *stable across seeds*; the deterministic variant has DDPG-like high variance.



<https://arxiv.org/abs/1801.01290>

- ▶ SAC v1 fixed many of DDPG's issues, but inherited one: it is brittle to the temperature hyperparameter α that controls exploration (and α must be tuned per task)
 - → automatic temperature tuning (SAC v2) [2]

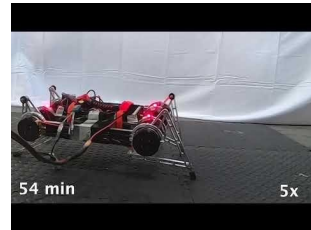
- Adaptive temperature coefficient

$$\sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} \left[r(\mathbf{s}_t, \mathbf{a}_t) + \underbrace{\alpha}_{\text{temperature}} \mathcal{H}(\pi(\cdot | \mathbf{s}_t)) \right]$$

- Extend to real-world tasks such as locomotion for a quadrupedal robot and robotic manipulation with a dexterous hand

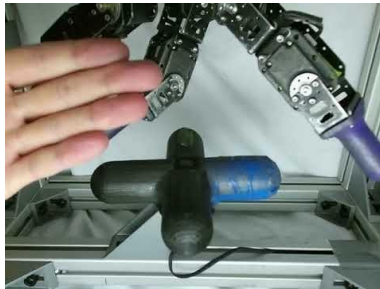
Minitaur quadruped learning forward locomotion end-to-end with SAC on *real hardware* [2].

- ▶ No simulator, no curriculum
- ▶ Video overlays: “54 min” = elapsed real-world training; “5×” = playback speed
- ▶ Walks reliably within ≈ 2 h of real-world experience



<https://arxiv.org/abs/1812.05905>

- ▶ **Task:** a multi-fingered hand opens a valve to a target angle, learning from raw joint sensors [2]
- ▶ End-to-end real-world learning in ≈ 20 hours wall-clock
- ▶ With valve angle exposed as input: **SAC 3 hours vs. PPO 7.4 hours** — the off-policy, entropy-regularised advantage shows clearly in sample efficiency



<https://sites.google.com/view/sac-and-applications>

Constrained problem — maximise return subject to a minimum policy entropy $\bar{\mathcal{H}}$ (so we set a *target entropy* instead of a temperature):

$$\max_{\pi_{0:T}} \mathbb{E}_{\rho_{\pi}} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right] \quad \text{s.t.} \quad \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_{\pi}} [-\log \pi_t(\mathbf{a}_t | \mathbf{s}_t)] \geq \bar{\mathcal{H}} \quad \forall t$$

Dual. Introduce a Lagrange multiplier $\alpha_t \geq 0$ per constraint (Levine [3]). Each inner problem is the usual MaxEnt objective with α_t as temperature; the multiplier itself minimises the dual:

$$\alpha_t^* = \arg \min_{\alpha_t \geq 0} \mathbb{E}_{\mathbf{a}_t \sim \pi_t^*} \left[-\alpha_t (\log \pi_t^*(\mathbf{a}_t | \mathbf{s}_t) + \bar{\mathcal{H}}) \right]$$

In practice — one SGD step on a single scalar, jointly with the actor/critic updates:

$$J(\alpha) = \mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta}} \left[-\alpha (\log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) + \bar{\mathcal{H}}) \right], \quad \alpha \leftarrow \alpha - \lambda_{\alpha} \hat{\nabla}_{\alpha} J(\alpha)$$

λ_{α} is just the learning rate for that one scalar — same Adam step size as the actor and critic.

SAC v2: The Same Loop, One Extra Line

Red = everything SAC v2 adds to the algorithm you already saw. α is now a *learned* scalar, not a hyperparameter.

$\lambda_Q, \lambda_\pi, \lambda_\alpha$ are **learning rates** (Adam step sizes) for the critic, the actor and the temperature — all 3×10^{-4} in the paper.

Initialize actor θ , critics ϕ_1, ϕ_2 ; targets $\phi_i^- \leftarrow \phi_i$; replay $\mathcal{D} \leftarrow \emptyset$; **initialize α ; set target entropy $\bar{\mathcal{H}} = -\dim(\mathcal{A})$**

for each iteration do

for each environment step do

$\mathbf{a}_t \sim \pi_\theta(\cdot | \mathbf{s}_t)$; $\mathbf{s}_{t+1} \sim p$; $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})\}$

end

for each gradient step do

 sample minibatch from $\mathcal{D}$; $\tilde{\mathbf{a}}_{t+1} \sim \pi_\theta(\cdot | \mathbf{s}_{t+1})$

$y = r_t + \gamma(Q_\phi^{\min}(\mathbf{s}_{t+1}, \tilde{\mathbf{a}}_{t+1}) - \alpha \log \pi_\theta(\tilde{\mathbf{a}}_{t+1} | \mathbf{s}_{t+1}))$

$\phi_i \leftarrow \phi_i - \lambda_Q \hat{\nabla}_{\phi_i} [\frac{1}{2}(Q_{\phi_i}(\mathbf{s}_t, \mathbf{a}_t) - y)^2]$, $i \in \{1, 2\}$

$\theta \leftarrow \theta - \lambda_\pi \hat{\nabla}_\theta J_\pi(\theta)$ (J_π critic term uses $Q_\phi^{\min}$)

$\alpha \leftarrow \alpha - \lambda_\alpha \hat{\nabla}_\alpha [-\alpha(\log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) + \bar{\mathcal{H}})]$

$\phi_i^- \leftarrow \tau \phi_i + (1 - \tau) \phi_i^-$, $i \in \{1, 2\}$

end

end

Read the new line: if the actor is more decisive than the target ($\log \pi_\theta > -\bar{\mathcal{H}}$), α is pushed *up* and exploration is bought back; if it is too random, α falls. One scalar, one gradient step, and the last brittle hyperparameter is gone.

Setting	Default (Haarnoja 2018b) [2]
Policy parameterization	Squashed Gaussian: $\mathbf{a} = \tanh(\mu_\theta + \sigma_\theta \odot \epsilon)$
Critic	Twin soft Q_{ϕ_1}, Q_{ϕ_2} ; clipped double- Q target
Target network update	Polyak: $\phi_i^- \leftarrow \tau \phi_i + (1 - \tau) \phi_i^-$, $\tau = 0.005$
Target entropy $\bar{\mathcal{H}}$	$\bar{\mathcal{H}} = -\dim(\mathcal{A})$ (one nat per action dim)
Optimizer	Adam, learning rate 3×10^{-4} for Q , π , and α
Discount γ	0.99
Replay buffer / batch	$ \mathcal{D} = 10^6$, batch size 256
Reward scale (v1 only)	task-specific (replaced by auto- α in v2)

These are the defaults that ship with most SAC implementations (SB3, rlpyt, the authors' code). Tune only $\bar{\mathcal{H}}$ and network sizes per task.

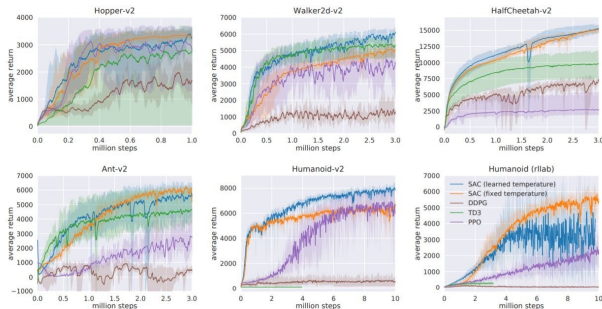


Figure 1: Training curves on continuous control benchmarks. Soft actor-critic (blue and yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

Figure 1: Training curves on continuous control benchmarks. Soft actor-critic (blue and yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

<https://arxiv.org/abs/1801.01290>

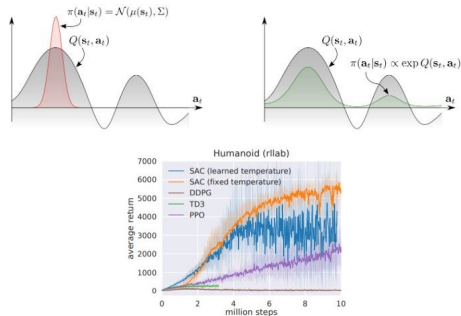
Task: Sawyer arm trained with SAC to stack a block.

- ▶ Evaluated under *perturbations unseen at training time* (a human pushes the block / arm)
- ▶ The stochastic, entropy-regularised policy adapts to off-distribution states
- ▶ A deterministic policy would freeze on the trained trajectory



<https://sites.google.com/view/sac-and-applications>

- ▶ Lack of experiments on hard-exploration problems (entropy is undirected — $\rightarrow$ the exploration lecture)
- ▶ Approximating a multi-modal Boltzmann distribution with a *unimodal* Gaussian collapses modes
- ▶ High-variance gradients when using automatic temperature tuning on hard tasks (e.g. Humanoid rllab)



Soft Actor-Critic: **Summary**

DQN + DPG → **DDPG** → +clipped double-Q → **TD3**
→ +max-entropy / stochastic actor → **SAC v1** → +automatic temperature → **SAC v2**

Algorithm	Off-policy?	Policy	Exploration	Stability
DDPG	Yes (replay)	Deterministic	Injected noise	Brittle
TD3	Yes (replay)	Deterministic	Injected noise	Improved
SAC v1	Yes (replay)	Stochastic	Entropy bonus	Robust (α tuned)
SAC v2	Yes (replay)	Stochastic	Entropy bonus	Robust (auto α)

- ▶ An **off-policy maximum-entropy** deep RL algorithm: sample-efficient, scales to high-dimensional observation/action spaces, robust to seeds and noise
- ▶ **“Soft” in one line:** every hard $\max_a Q$ is replaced by $\alpha \log \sum_a \exp(Q/\alpha)$ — equivalently, the greedy policy becomes the softmax policy $\pi \propto \exp(Q/\alpha)$, and the objective picks up $+\alpha\mathcal{H}$. Set $\alpha \rightarrow 0$ and you are back to standard RL
- ▶ **Theory:** soft policy iteration is ordinary policy iteration on the reward $r + \alpha\mathcal{H}$; the SAC objectives are its function-approximation instantiation
- ▶ **Algorithm:** soft critic (clipped double-Q) + reparameterised stochastic actor + automatic temperature — everything from DDPG/TD3 reused
- ▶ **Empirically:** outperforms DDPG, PPO, TD3 and Soft Q-Learning in optimality, sample complexity and robustness

- ▶ SAC (2018) is *still* the de facto default for continuous control in simulation and real-world robotics
- ▶ Direct descendants improved sample efficiency 5–10 $\times$:

Method	Idea
TQC (2020) [9]	truncated quantile critics — controls overestimation
REDQ (2021) [10]	randomized ensemble of Q -functions, high update-to-data ratio
DroQ (2021) [11]	dropout-regularized critics — REDQ's efficiency, fewer nets

- ▶ SAC's entropy bonus *is* exploration via a stochastic policy — but it is undirected
- ▶ **Exploration:** directed strategies beyond entropy
 - UCB / optimism, count-based bonuses, curiosity / intrinsic motivation
 - the answer for hard-exploration, sparse-reward tasks SAC struggles on
- ▶ **Further out:**
 - **Offline RL:** on a fixed buffer the entropy bonus does *not* save you — SAC suffers DDPG's OOD-action extrapolation error (motivates BCQ/CQL)
 - **Real-world robot learning:** SAC is the workhorse for from-scratch robots (quadrupeds, dexterous hands) — revisited there
- ▶ Takeaway: entropy is *one* answer to exploration; the next lecture covers the rest

Soft Actor-Critic: **References**

- [1] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018).
Soft Actor-Critic: Off-Policy Maximum Entropy Deep RL with a Stochastic Actor (SAC v1).
ICML 2018, PMLR 80, 1861–1870; arXiv:1801.01290.
- [2] Haarnoja, T., Zhou, A., Hartikainen, K., et al. (2018).
Soft Actor-Critic Algorithms and Applications (SAC v2 / automatic temperature).
arXiv:1812.05905.
- [3] Levine, S. (2018).
Reinforcement Learning and Control as Probabilistic Inference: Tutorial and Review.
arXiv:1805.00909.
- [4] Lillicrap, T. P., Hunt, J. J., Pritzel, A., et al. (2015).
Continuous Control with Deep Reinforcement Learning (DDPG).
arXiv:1509.02971; *ICLR 2016*.
- [5] Fujimoto, S., van Hoof, H., & Meger, D. (2018).
Addressing Function Approximation Error in Actor-Critic Methods (TD3).
ICML 2018, PMLR 80, 1587–1596; arXiv:1802.09477.

- [6] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015).
Human-Level Control through Deep Reinforcement Learning (DQN).
Nature, 518(7540), 529–533.

- [7] Haarnoja, T., Tang, H., Abbeel, P., & Levine, S. (2017).
Reinforcement Learning with Deep Energy-Based Policies (Soft Q-Learning).
ICML 2017, PMLR 70, 1352–1361; arXiv:1702.08165.

- [8] Levine, S., Pastor, P., Krizhevsky, A., & Quillen, D. (2016).
Learning Hand-Eye Coordination for Robotic Grasping with Deep Learning and Large-Scale Data Collection.
arXiv:1603.02199.

- [9] Kuznetsov, A., Shvechikov, P., Grishin, A., & Vetrov, D. (2020).
Controlling Overestimation Bias with Truncated Mixture of Continuous Distributional Quantile Critics (TQC).
ICML 2020, PMLR 119, 5556–5566; arXiv:2005.04269.

- [10] Chen, X., Wang, C., Zhou, Z., & Ross, K. (2021).
Randomized Ensembled Double Q-Learning: Learning Fast Without a Model (REDQ).
ICLR 2021; arXiv:2101.05982.
- [11] Hiraoka, T., Imagawa, T., Hashimoto, T., Onishi, T., & Tsuruoka, Y. (2021).
Dropout Q-Functions for Doubly Efficient Reinforcement Learning (DroQ).
ICLR 2022; arXiv:2110.02034.
- [12] Garg, A. CSC2621: Reinforcement Learning in Robotics, University of Toronto.
<https://animesh.garg.tech/CSC2621-W20/>

These slides have been adapted from

- ▶ Animesh Garg, [CSC2621: Reinforcement Learning in Robotics](#), University of Toronto **[12]**
- ▶ Primary papers: Haarnoja 2018a **[1]** (SAC v1), Haarnoja 2018b **[2]** (SAC v2), Levine 2018 **[3]** (control as inference)